

Платформа Codacy, Новые возможности,
Искусственный интеллект в разработке программного обеспечения

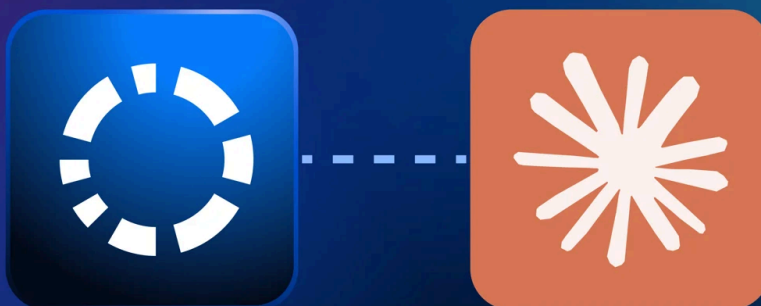
09/06/2026

Представляем Codacy Skills (часть 1): разблокируйте пул-реквесты с помощью одного запроса



Александр Риццо

9 мин на чтение



Более 15 000 организаций используют Codacy для проверки пул-реквестов на соответствие заранее установленным стандартам качества, безопасности и искусственного интеллекта. Эта обратная связь всегда была доступна в разделе PR: обнаруженные проблемы с качеством и безопасностью, снижение тестового покрытия, пробелы в намерениях и предложения по исправлению.

Это хорошо работало до появления искусственного интеллекта. Теперь,

когда агенты-кодеры увеличили количество написанного кода в разы, проверка кода стала узким местом для команд, внедряющих агентские рабочие процессы. Сканирование по-прежнему выполняется автоматически, но для сортировки результатов требуется больше усилий со стороны человека: нужно устранять критические проблемы, открывать тикеты в Jira, чтобы отложить важные, но не срочные исправления, и помечать ложные срабатывания как игнорируемые.

Сегодня мы представляем **Codacy Skills** — новый способ, с помощью которого такие кодинг-агенты, как Клод, могут выполнять рутинную работу по разблокировке пул-реквестов, настройке правил и параметров Codacy и даже локальному анализу кода перед коммитом.

По сути, Codacy Skills учат кодинг-агентов использовать наш новый [Codacy Cloud CLI](#) и Analysis CLI для решения целого ряда сложных (смею ли я сказать «революционных»?) задач. Чтобы помочь вам начать работу, мы решили создать серию мини-блогов, в которых подробно разберем каждый навык. Сегодня мы начинаем с 'Часть 1: установите Codacy Cloud CLI для своего агента'.

установки Codacy Cloud CLI и навыков

The **Codacy Cloud CLI** ([@codacy/codacy-cloud-cli](#)) выводит данные Codacy в терминал: проблемы, уязвимости в системе безопасности, результаты анализа пул-реквестов, показатели покрытия кода, настроенные инструменты и шаблоны для GitHub, GitLab и Bitbucket. По умолчанию вывод представляет собой таблицу, но при необходимости его можно перенаправить куда-либо в формате JSON.

Это проект с открытым исходным кодом, который можно установить через npm. После установки войдите в систему, чтобы подключить свою учетную запись [codacy.com](#).

Вот что вы увидите в терминале:

```
[[base) /Users/sergey@klaus ~/ % sudo npm install -g @codacy/codacy-cloud-cli ]
changed 42 packages in 797ms

18 packages are looking for funding
run `npm fund` for details
[[base) /Users/sergey@klaus ~/ % codacy login ]

Codacy Login

You need an Account API Token to authenticate.
Get one at: https://app.codacy.com/account/access-management
My Account > Access Management > API Tokens

[API Token: *****]
✓ Logged in as Klaus (klaus@klaus.com)
Token stored at /Users/sergey@klaus/.codacy/credentials
(base) /Users/sergey@klaus ~/ % █
```

Навык **Codacy Cloud CLI** помогает агентам по написанию кода использовать Codacy Cloud CLI. Они работают с Claude Code, OpenAI Codex, GitHub Copilot и Gemini CLI в соответствии со стандартом Agent Skills.

Примечание: вы можете использовать Codacy Cloud CLI как с установленными навыками Codacy, так и без них. Если вы хотите использовать Cloud CLI вручную, [ознакомьтесь с нашей документацией](#), где приведены подробные инструкции, команды и примеры рабочих процессов. Вы также можете [встроить Codacy Cloud CLI в свою среду непрерывной интеграции](#) для расширенной автоматизации рабочих процессов.

Use the snippet below to add the marketplace and install the Codacy Skills plugin for Claude ([see instructions for other agents here](#)):

Once installed successfully, you will see this:



```
[[base) ~ % claude plugin marketplace add codacy/codacy-skills ]
SSH not configured, cloning via HTTPS: https://github.com/codacy-skills.git
18Refreshing marketplace cache (timeout:120s)_
Cloning repository (timeout 120s): https://github.com/codacy/codacy-skills.git
Clone complete, validating marketplace...
Cleaning up old marketplace cache...
✓ Successfully added marketplace: codacy (declared in user settings)
[[base) ~ % claude plugin install codacy-skills@codacy ]
✓ Successfully installed plugin: codacy-skills@codacy (scope: user)
(base) ~ % █
```

Both the Codacy Cloud CLI and Codacy Skills are free to install on every Codacy plan. What the agent can act on still follows the Codacy features available on your plan ([see our pricing page for more details](#)).

You can now perform basic operations like adding your repositories to Codacy.

```
opencode — * Add repository to Codacy — claude — 108x42

Welcome back klauz!



Sonnet 4.6 (1M context) · Claude Pro ·
Organization
~/Documents/GitHub/opencode

Tips for getting started
Run /init to create a CLAUDE.md file with instruct...

What's new
Renamed `/simplify` to `/code-review` with an opti...
Auto mode no longer suppresses `AskUserQuestion` w...
Fixed Windows PowerShell tool failing with "comman...
/release-notes for more

> add this repo to codacy

• Skill(codacy-skills:codacy-cloud-cli)
  ↳ Successfully loaded skill

• Let me check the git remote to find the org and repo name.

• Bash(git remote -v)
  ↳ origin      https://github.com/[redacted]/opencode (fetch)
  ↳ origin      https://github.com/[redacted]/opencode (push)
  ↳ upstream    https://github.com/sst/opencode (fetch)
  ↳ upstream    https://github.com/sst/opencode (push)

• The repo is [redacted]/opencode on GitHub. Now I'll add it to Codacy.

• Bash(codacy repository gh [redacted] opencode --add)
  ↳ - Adding opencode to Codacy...
  ↳ ✓ Repository opencode added to Codacy.
  ↳ Note: the repository will be available after a few minutes, once the initial cloning and analysis
  ↳ is complete.

• The repository [redacted]/opencode has been added to Codacy. The initial cloning and analysis will run
  in the background – it should be available on your Codacy dashboard in a few minutes.

* Churned for 15s

> █

? for shortcuts · ← for agents
```

Now, let's get to the interesting bit. Below are our three favorite ways to use the `codacy-cloud-cli` skill like a pro.

Use case 1: Clearing a blocked PR in one prompt

Here is the case we built this for: a PR is failing the merge check. You can set up to six criteria to trigger your [Codacy PR gate](#):

- number of new issues introduced
- number of security issues introduced
- hitting the complexity threshold
- hitting the duplication threshold

insufficient diff coverage (percentage of changed lines of code that are covered by tests)

overall test coverage drops.

Whichever is triggered, the PR comes back red. Normally you would open each finding, fix it, write the missing test, and re-run the analysis.

With the `codacy-cloud-cli` skill installed, you can hand the whole thing to Claude Code in one prompt:

```
Pull request 42 is failing the Codacy gate.  
Fix what's real, add the tests it needs,  
ignore the false positives with a reason,  
then re-run the scan.
```

The agent pulls the analysis and reads back everything that's blocking the gate, using the `pull-request` subcommand to return the annotated diff with new issues inline and uncovered lines marked.

From there the agent fixes the genuine issues (applying Codacy's suggested fix where there is one, writing the rest itself), writes missing tests, refactors any duplicated blocks, dismisses the confirmed false positives with a logged reason, and re-runs the analysis to confirm the gate is green.

Quick disclaimer: The PR check may not go green on the first pass, but instead of scattering your code review and remediation efforts between agent, terminal, IDE and browser tabs, you get to triage your scan results quickly from a single place, in bulk, and against your existing coding standards.

Use case 2: Do a security sweep across the repo

The same pattern scales past a single pull request. Point the agent at the backlog:

```
Fix the critical and high security findings  
in this repo.
```

Your agent runs the lookup and reads the findings with their severity and CVE context, using:

Findings can also be filtered by scan type (SAST, Secrets, SCA, IaC), status (Overdue, Due soon, On track). This allows you to pull detailed vulnerability reports and instant, scoped-out security audits.

```

* Review Codacy issues — claude — 104x71

• Here's a summary of your Codacy issues for codacy-issues:

---
Issues Overview

| Category        | Count      |
|-----------------|------------|
| ErrorProne      | 3,727      |
| BestPractice    | 2,661      |
| CodeStyle       | 1,540      |
| Complexity      | 1,293      |
| UnusedCode      | 592        |
| <b>Security</b> | <b>531</b> |



| Severity | Count |
|----------|-------|
| High     | 4,746 |
| Warning  | 4,207 |
| Info     | 1,559 |
| Error    | 49    |



Almost all issues (~98%) are in TypeScript files.

---
Critical Security Issues (top priority)

| # | Issue                                                   | File                                                   |
|---|---------------------------------------------------------|--------------------------------------------------------|
| 1 | XSS — <code>innerHTML</code> with user-controlled data  | <code>packages/ui/src/v2/components/icon.tsx:61</code> |
| 2 | Timing attack — token comparison via <code>!=="</code>  | <code>packages/app/src/pages/layout.tsx:750</code>     |
| 3 | Hardcoded secret — Stripe Publishable Key in script     | <code>packages/console/app/package.json:9</code>       |
| 4 | SSRF — user-controlled URL passed to <code>fetch</code> | <code>sdks/vscode/src/extension.ts:94</code>           |
| 5 | SSRF — user-controlled URL passed to <code>fetch</code> | <code>script/github/close-prs.ts:296</code>            |

Notable High Security Issues

- Generic Object Injection Sink in multiple files (session.tsx:315, message-timeline.tsx:374, session-composer-state.ts:53)
- target="_blank" without rel="noopener" — packages/console/app/src/routes/index.tsx:721
- chmodSync with non-literal argument — packages//script/postinstall.mjs:155

---
Would you like me to dive into any specific category, fix the critical security issues, or look at the issues on a specific branch/PR?

* Crunched for 33s
```

Use case 3: Triage false positives, in bulk

False positives are where teams reviewing high volumes of AI-generated code lose the most time. Instead of dismissing them one by one in the UI, the agent

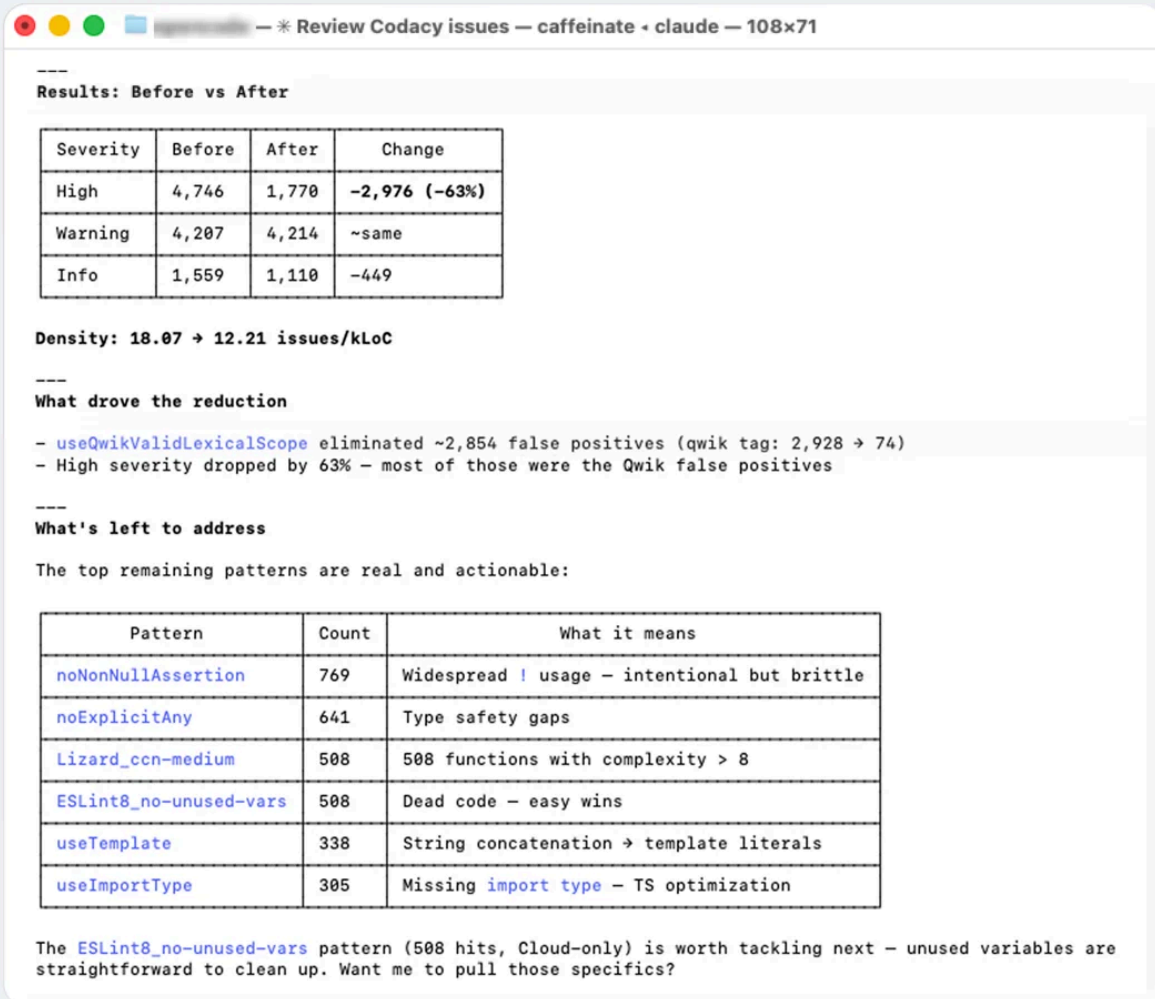
clears them in a single command, with a reason attached:

```
Ignore all issues that are flagged as false  
positives and tag each with an ignore reason
```

Codacy flags which findings are likely false positives (available on Codacy's [Business plan](#)), so the agent acts on Codacy's data rather than guessing. Here's the Cloud CLI command it uses:

The reasons it attaches are logged, and become a record of why each call was made, which is useful feedback the next time you tune the repo's rule configuration (we'll talk more about configuring Codacy rules in Part 2 of our Skills series, so stay tuned).

In the example below, this one prompt helped us reduce the issue density from 18.07 to 12.21 issues/kLOC.



One practical consequence (for your wallet)

Let me be precise about the division of labor: **Codacy does not edit your code.** Your agent does, using Codacy as the source of truth for what needs attention and as the check that the change actually worked.

One practical consequence: **the analysis itself consumes no AI tokens.**

Codacy's pull request review, including the AI Reviewer layer, is already included in every paid plan at a flat per-seat price. Your agent only consumes tokens for the edits you ask for, and nothing on the analysis underneath.

What's next

The static analysis, security scanning, and coverage tracking are the same as they were before Codacy Skills.

What is new is where they reach the developer: the terminal and the agent they are already working with. For a team leaning on AI coding tools, that is the difference between a quality system people have to remember to visit and one that travels with the work.

This is the first post in our new mini-blog-series on Codacy Skills. Stay tuned for more over the coming weeks, as we will get into more advanced use cases like scanning and fixing code locally before you push, and auto-tuning your repo configuration to cut out noise and get more accurate results.

To get started, install the CLI and the skills from the [Codacy Cloud CLI documentation](#), or [read the source on GitHub](#).

Related blog posts

The Complete Guide to Technical Debt Tracking for Engineering Leaders

Every engineering team knows technical debt exists. Far fewer can tell you exactly where it lives, how much it costs, or whether it is growing.



Introducing AgentLinter: Codacy now scans your AI agent config files

Most engineering teams now have AI agent config files sitting in their repos. CLAUDE.md tells Claude Code how to behave, while AGENTS.md does the same...



Introducing Container Image Scanning: Secure your images against new CVEs

A vigilant engineering team today runs seven or eight tools to cover risk and security vulnerabilities. Code quality in one place, SAST in another, SCA...



Subscribe to our blog

Stay updated with our monthly newsletter.

Enter Your Email*

Subscribe

Platform

Code Quality

Security

Coverage

AI Reviewer

AI Risk Hub

AI Guardrails

Resources

Blog

Resources

Documentation

AI Giants podcast

Why Codacy

About us

[Customer Stories](#)

[Codacy vs Sonarqube](#)

[Codacy vs Snyk](#)

[Codacy vs CodeRabbit](#)

[Pricing](#)

[About Codacy](#)

[Careers](#)

[Cookies](#)

[Privacy Policy](#)

[Security Policy](#)

[Terms of Service](#)

Contact us

[Send an email](#)

[Chat with us](#)



SOC2 Compliant



GDPR Compliant

Lisbon, Portugal



©2026 Copyright. All Rights Reserved.